

Report

Ich würde die Streuung bei Namen anhand der Häufigkeit und Anzahl der unterschiedlichen Vor- und Nachnamen berechnen. In der Realität kommen Namen natürlich unterschiedlich häufig vor. Sie sind also nicht gleichmäßig verteilt. Sollte die Faker Bibliothek saubere zufällige Datensätze (unrealistisch) liefern, müssten also alle Vornamen bzw. alle Nachnamen (annähernd) gleich oft vorkommen.

Die hohe Zufälligkeit der Streuung der erzeugten Namen würde sich anhand einer hohen Anzahl unterschiedlicher Namen, die allerdings alle ähnlich häufig vorkommen, zeigen. Das wäre realitätsfern.

Ich habe die Streuung der Bibliothek überprüft, indem ich folgende Daten ausgelesen habe:

Wie viele unterschiedliche Vornamen gibt es? $\approx$ 2002

Wie viele unterschiedliche Nachnamen gibt es? $\approx$ 404

Wie oft kommt der häufigste / seltenste Vorname vor und wie oft kommt ein Vorname durchschnittlich vor? $\approx$ MAX = Jonas 492 MIN = Natalja 199 AVG = 249.75

Wie oft kommt der häufigste / seltenste Nachname vor und wie oft kommt ein Nachname durchschnittlich vor? $\approx$ MAX = Seifert 2533 MIN = Tintzmann 1123 AVG = 1237.62

Anhand dieser Daten kann ich sagen, dass der häufigste Vorname 2,47 mal so oft vorkommt wie der Seltenste und der häufigste Nachname 2,26 mal so oft vorkommt wie der Seltenste. Das entspricht einer realistisch ungleichmäßigen Verteilung und ist somit realitätsnah.

Um die Performance der Datenabfrage zu überprüfen, habe ich folgenden Select verwendet:

“SELECT COUNT(DISTINCT vorname) FROM personen;“

Performance ohne Index: Run Time: real 0.305, user 0.281250, sys 0.031250

Performance mit Index: Run Time: real 0.057, user 0.062500, sys 0.000000

Durch das Verwenden des Index ist die Abfrage ca. 5 mal schneller erfolgt.

Speicherbedarf ohne Index: Seitengröße: 4096B Seitenanzahl: 2870 4096 *
2870 = 11.755520MB

Speicherbedarf mit Index: Seitengröße: 4096B Seitenanzahl: 6703 4096 *
6703 = 27.455488MB

Der Speicherbedarf hat sich also um den Faktor 2.34 erhöht.

Zum Vergleich habe ich eine Datenbank erstellt, in der 50% der Personen den Vornamen “Max” haben.

Performance ohne Index: Run Time: real 0.200, user 0.156250, sys 0.031250

Performance mit Index: Run Time: real 0.064, user 0.046875, sys 0.000000

Speicherbedarf ohne Index: Seitengröße: 4096B Seitenanzahl: 2648 4096 *
2648 = 10.846208MB

Speicherbedarf mit Index: Seitengröße: 4096B Seitenanzahl: 4336 4096 *
4336 = 17.760256MB

Die Performance ist ca. 3 mal besser, der Speicherbedarf ist um den Faktor 1,64 höher.

Auffällig ist, dass der Select bei der indexierten Biased-Datenbank viel weniger Speicher benötigt und dabei fast die gleiche Performance bietet wie die indexierte “normale” Datenbank. Ich denke, das ist darauf zurückzuführen, dass der Wert des Schlüssels “Vorname” in 50% der Fälle gleich ist. Somit müssen weniger “Knoten” beim Erstellen des Index beachtet werden, was zu einem Index mit weniger Seiten führt.